

Frontend Engineering: Day 3

JS Introduction, Variables, Declaration, Hoisting

Sayan Chowdhury

Engineer- Data Analytics

Design Automation

Strict Mode

The purpose of "**use strict**" is to indicate that the code should be executed in "**strict mode**". With strict mode, you can not use undeclared variables.

Strict mode is declared by adding "use strict"; to the beginning of a script or a function. Declared at the beginning of a script, it has global scope (all code in the script will execute in strict mode):



```
"use strict";  
x = 3.14; .....// This will cause an error because x is not declared
```



```
"use strict";  
myFunction();  
function myFunction(){  
  y = 3.14; .....// This will also cause an error because y is not declared  
}
```

JavaScript Data Structures

Built-in Data Structures (The "Big Four")

Arrays

The most common data structure. It is an ordered list of values. In JS, arrays are dynamic (they can grow/shrink) and can hold multiple data types.

Objects

Used to store keyed collections (key-value pairs). It is the foundation of almost everything in JavaScript.

Maps (ES6+)

Similar to Objects, but with key differences:
Keys can be any type (functions, objects, numbers).
They maintain the insertion order.
They have a size property.

Sets (ES6+)

A collection of unique values. No duplicates are allowed.



```
const fruits = ["Apple", "Mango", "Banana"];
```



```
const user = { name: "Sayan", age: 25 };
```



```
const map = new Map();  
map.set('1', 'str1'); // a string key  
map.set(1, 'num1'); // a numeric key
```



```
const mySet = new Set([1, 2, 2, 3]); // Result: {1, 2, 3}
```

JavaScript Operators

Operators are for Mathematical and Logical Computations

- The Assignment Operator **=** assigns values
- The Addition Operator **+** adds values
- The Multiplication Operator ***** multiplies values
- The Comparison Operator **>** compares values

Operator	Example	Same As
=	x = y	x = y
+=	x += y	x = x + y
-=	x -= y	x = x - y
*=	x *= y	x = x * y
/=	x /= y	x = x / y
%=	x %= y	x = x % y
**=	x **= y	x = x ** y

Operator	Description
+	Addition
-	Subtraction
*	Multiplication
**	Exponentiation
/	Division
%	Modulus (Division Remainder)
++	Increment
--	Decrement

JavaScript Operators

Operator	Description	Example
==	equal to	x == 5
===	equal value and equal type	x === 5
!=	not equal	x != 5
!==	not equal value or not equal type	x !== 5
>	greater than	x > 5
<	less than	x < 5
>=	greater than or equal to	x >= 5
<=	less than or equal to	x <= 5

The Spread Operator (...)

The ... operator splits an array into individual elements.

Operator	Description
&&	logical and
	logical or
!	logical not

```
// The ... operator can be used to join arrays:~  
const arr1 = [1, 2, 3];~  
const arr2 = [4, 5, 6];~  
const arr3 = [...arr1, ...arr2];~  
// The ... operator can pass array arguments to functions:~  
const numbers = [23, 55, 21, 87, 56];~  
let minValue = Math.min(...numbers);~  
let maxVlue = Math.max(...numbers);~  
// The ... operator can be used to copy arrays:~  
const arr1 = [1, 2, 3];~  
const arr2 = [...arr1];~
```

JavaScript Operators

Comparing data of different types may give unexpected results.

When comparing a string with a number, JavaScript will convert the string to a number when doing the comparison. An empty string converts to 0. A non-numeric string converts to **NaN** which is always false.

Case	Value
2 < 12	true
2 < "12"	true
2 < "L&T"	false
2 > "L&T"	false
2 == "L&T"	false
"2" < "12"	false
"2" > "12"	true
"2" == "12"	false

What will be the output of below?

2===“2”

True/False?

JavaScript Conditionals

Conditional statements include:

- *if*
- *if...else*
- *if...else if...else*
- *switch*
- *ternary (?:)*

Use **if** , if a specified condition is true

Use **else**, if the same condition is false

Use **else if**, to specify a new condition to test, if the first condition is false

Use **switch** to specify many alternative code blocks to be executed

Use **(?:)** (ternary) as a shorthand for if...else

```
if (condition) {  
    //code to execute if the condition is true  
}
```

```
if (condition) {  
    //code to execute if the condition is true  
} else {  
    //code to execute if the condition is false  
}
```

```
if (condition1) {  
    //code to execute if condition1 is true  
} else if (condition2) {  
    //code to execute if the condition1 is false and condition2 is true  
} else {  
    //code to execute if the condition1 is false and condition2 is false  
}
```

JavaScript Conditionals



```
switch(expression) {  
  case x:  
    // code block  
    break;  
  case y:  
    // code block  
    break;  
  default:  
    // code block  
}
```



```
condition ? expression1 : expression2
```



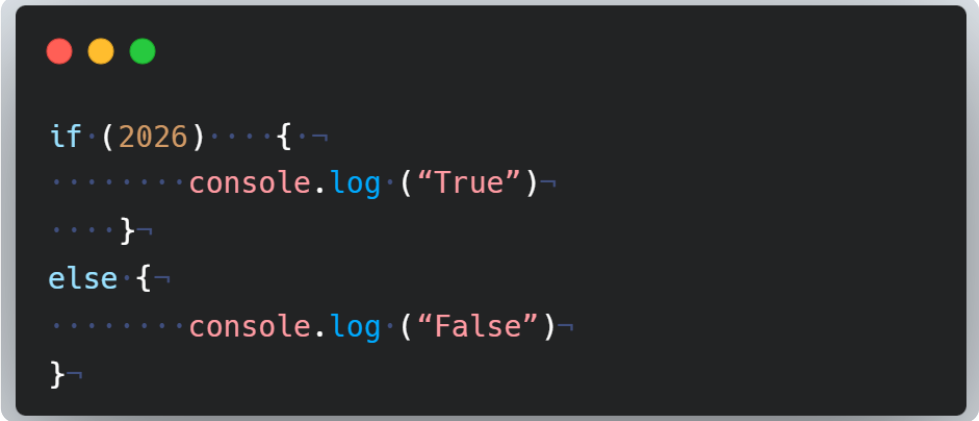
```
let day = "Monday";  
  
switch(day) {  
  case "Monday":  
    console.log("Today is Monday. Start of the workweek!");  
  case "Thursday":  
  case "Friday":  
    console.log("It's a weekday. Keep working!");  
    break;  
  case "Saturday":  
  case "Sunday":  
    console.log("It's the weekend. Time to relax!");  
    break;  
  default:  
    console.log("Invalid day. Please enter a valid day.");  
}
```



```
let age = 18;  
let canVote = age >= 18 ? "You can vote!" : "You cannot vote.";  
  
console.log(canVote); // Output: "You can vote!"
```


JavaScript Conditionals

What will be the output of below code?



```
if (2026) {  
    console.log("True")  
}  
else {  
    console.log("False")  
}
```

True/False?

JavaScript Conditionals

The Nullish Coalescing Operator (??)

The `??` operator returns the right operand when the left operand is nullish (null or undefined), otherwise it returns the left operand.



```
let name = null;
let text = "missing";
let result = name ?? text;
console.log(result)
```

JavaScript Loops

The For Loop:

The for statement creates a loop with 3 optional expressions:

```
for (expr1; expr2; expr3) {  
  // code block to be executed  
}
```

- **exp1** is executed one time before the execution of the code block.
- **exp2** defines the condition for executing the code block.
- **exp3** is executed every time the code block has been executed.

exp1, exp2, exp3 is optional based on how you write the code

```
for (let i = 0; i < 5; i++) {  
  text += "The number is " + i + "<br>";  
}
```

Loop Scope

```
let i = 5;  
for (i = 0; i < 10; i++) {  
  // some code  
}  
// Here i is 10
```

```
let i = 5;  
for (let i = 0; i < 10; i++) {  
  // some code  
}  
// Here i is 5
```

In the 1st example, let i = 5; is declared outside the loop. In the 2nd example, let i = 0;, is declared inside the loop. When a variable is declared with let or const inside a loop, it will only be visible within the loop.

JavaScript Loops

The While Loop

The while loop loops through a block of code as long as a specified condition is true.

```
while (condition) {  
  // code block to be executed  
}
```

Jump Statements

Jump statements let you change the flow abruptly using:

- **break** - exits a loop or switch
- **continue** - skips the current loop iteration
- **return** - exits from a function
- **throw** - jumps to error handling



```
while (i < 10) {  
  text += "The number is " + i;  
  i++;  
}
```



```
for (let i = 0; i < 10; i++) {  
  if (i === 3) { break; }  
  text += "The number is " + i + "<br>";  
}
```



```
for (let i = 1; i < 10; i++) {  
  if (i === 3) { continue; }  
  text += "The number is " + i + "<br>";  
}
```

JavaScript Function

Functions are defined with the *function* keyword:

```
function name( p1, p2, ... ) {  
  // code to be executed  
}
```

- followed by the function name
- followed by parentheses ()
- followed by brackets { }
- The function name follows the naming rules for variables.
- Optional parameters are listed inside parentheses: (p1, p2, ...)
- Code to be executed is listed inside curly brackets: { }
- Functions can return an optional value back to the caller.

Local Variables

Variables declared within a JavaScript function, become LOCAL to the function.

Local variables can only be accessed from within the function.

```
function multiply(a, b) {  
  return a * b;  
}
```

```
// code here can NOT use carName  
function myFunction() {  
  let carName = "Volvo";  
  // code here CAN use carName  
}  
// code here
```

JavaScript Function

Function Invocation

The code inside a function is NOT executed when the function is defined.

The code inside a function will execute when “**something**” invokes the function:

- When it is called from JavaScript code
- When an event occurs (e.g. a user clicks a button)
- Automatically (self invoked)



```
function sayHello(){  
  return "Hello World";  
}  
  
sayHello();
```



```
function sayHello(){  
  return "Hello World";  
}  
  
let greeting = sayHello();
```



```
function sayHello(){  
  return "Hello World";  
}  
  
console.log(sayHello());
```



```
// Convert Fahrenheit to Celsius:  
function toCelsius(fahrenheit){  
  return (5/9) * (fahrenheit-32);  
}  
  
// Call => toCelsius() function  
let value = toCelsius(77);
```

JavaScript Function

Will this code run error free?



```
const times = multiply(5,4)
console.log(times)

function multiply(a, b) {
  return a * b;
}
```

What is the difference between
Arguments and *Parameters*?

JavaScript Function

Functions Can Be Called from Anywhere

You can call functions from other functions, from events, or from any code block.

```
function sayHello() {  
  return "Hello World";  
}  
  
function showHello() {  
  document.getElementById("demo").innerHTML +=  
    sayHello();  
}
```

Functions with Parameters

Parameters allow you to pass (send) values to a function.
Parameters are listed **inside the parentheses** in the function **definition**.

```
function multiply(a, b) {  
  return a * b;  
}  
  
let result = multiply(4, 5);
```

Parameters vs. Arguments

Parameters are the names listed in the function definition.
Arguments are the real values passed to, and received by the function..

```
function toCelsius(fahrenheit) {  
  return (5/9) * (fahrenheit - 32);  
}  
  
let value = toCelsius(5);
```

fahrenheit is parameter (points to `fahrenheit`)

5 is argument (points to `5`)

JavaScript Function

JavaScript Arrow Functions

Arrow Functions allow a shorter syntax for **function expressions**. You can skip the **function** keyword, the **return** keyword, and the **curly brackets**.

Just like *Lambda* in Python :)

Why bother using Arrow function?

Shorter Syntax

If the function body contains only one statement:

You can remove the word function, the curly brackets and the return keyword. Arrow function returns by default.



```
const add = (a, b) => a * b;
```



```
const add = (a, b) => {  
  return a + b;  
};
```



```
// Not using Arrow Function  
const add =  
  .....function(a, b) {  
    .....return a * b;  
    .....}  
// Using Arrow Function  
const add = (a, b) => a * b;
```

JavaScript



What is this?

Bike —————→ *const Bike*

What is the colour of this bike?

Red —————→ *Bike.color= red*

Brand of the Bike?

Ducati —————→ *Bike.brand= Ducati*

Name of this Bike?

Panigale —————→ *Bike.name= Panigale*

Bike.start()

Bike.stop()

Bike.brake()

Bike.neverstop()

JavaScript Objects

Objects are variables that can store both values and functions.

Values are stored as **key:value** pairs called properties.

Functions are stored as **key:function()** pairs called methods.

How to Create a JavaScript Object?

Objects in JavaScript Works like a Python Object but looks like a python dictionary. You should declare objects with the **const** keyword.

Using the new Keyword

You can create a new JavaScript object using **new Object()**

There is no need to use **new Object()**.

For readability, simplicity and speed, use an object literal instead.

```
const bike = {  
  type: "Ducati",  
  model: "Panigale",  
  color: "Red"  
};
```

```
// Create an Object  
const bike = {};  
  
// Add Properties  
bike.name = "Ducati";  
bike.model = "Panigale";  
bike.color = 'red';  
bike.mileage = 11;
```

```
const person = new Object({  
  firstName: "Abhijit",  
  lastName: "Sen",  
  age: 50,  
  eyeColor: "blue"  
});
```

JavaScript Objects

Object Properties

You can access object properties in two ways:

- Dot notation
- Bracket notation

```
//Dot Notation~  
objectName.propertyName~  
person.firstName;~
```

```
//Bracket Notation~  
objectName["propertyName"]~  
person["firstName"];~
```

JavaScript Object Methods

Object **methods** are function definitions stored as property values

```
const person = {~  
  firstName: "Niraj",~  
  lastName: "Nair",~  
  age: 50,~  
  fullName: function() {~  
    return this.firstName + " " +~  
    this.lastName;~  
  }~  
};~
```

JavaScript Objects: this

The *this* Keyword:

In an object method, **this** refers to the object.

In the example, **this** refers to the person object.
this.id means the id property of the person object.

Why Use this?

The this keyword makes it possible to use the same method with different objects.

this Alone?

When used alone, this refers to the global object.
In a browser, the global object is the **window** object.

```
const person = {  
  firstName: "Govind",  
  lastName: "Gopinath",  
  id: 5566,  
  getId: function() {  
    return this.id;  
  }  
};  
  
let number = person.getId();
```

```
let x = this;  
document.getElementById("demo").innerHTML = x;
```

JavaScript Objects Display

How to Display JavaScript Objects?

Displaying a JavaScript object will output **[object Object]**.

Why do you See [object Object]?

[Object Object] appears when you try to put an object into a context where a string is expected.

[Object Object] how JavaScript deals with this situation.

Some solutions to display JavaScript objects are:

Displaying the Object Properties by name

Displaying the Object Properties in a Loop

Displaying the Object using **Object.values()**

Displaying the Object using **JSON.stringify()**

You must use **person[x]** in the loop.

person.x will not work (Because x is the loop variable).

```
const person = {  
  name: "Steve",  
  age: 30,  
  city: "New York"  
};  
// Display Object  
console.log(person);
```

```
// Create an Object  
const person = {  
  name: "Steve",  
  age: 30,  
  city: "New York"  
};  
// Add Properties  
let text = person.name + ", " + person.age + ", " + person.city;
```

```
// Create an Object  
const person = {  
  name: "Steve",  
  age: 30,  
  city: "New York"  
};  
// Build a Text  
let text = "";  
for (let x in person) {  
  text += person[x] + ", ";  
}
```

JavaScript Loops

The for...in Loop

The for...in loop iterates over the enumerable properties of an object. It is typically used for iterating over object keys.

The for...of Loop

The for...of loop iterates over Array, Set, String. It is typically used for iterating over object keys.

The forEach()

The **forEach()** method calls a function for each element in an array. The **forEach()** method is not executed for empty elements.

array.forEach(function(currentValue, index, arr), thisValue)

```
for (key in object) {  
  //code block to be executed  
}
```

```
const person = {fname:"Steve", lname:"Jobs", age:25};  
let text = "";  
for (let x in person) {  
  text += person[x];  
}
```

```
const letters = ["a", "b", "c"];  
for (const x of letters) {  
  //code block to be executed  
}
```

```
let sum = 0;  
const numbers = [65, 44, 12, 4];  
numbers.forEach(myFunction);  
function myFunction(item) {  
  sum += item;  
}
```


Quiz Time!

What is the output?



```
"use strict";  
x = 10;  
console.log(x);
```

Ans: Throws ReferenceError: x is not defined (undeclared var in strict mode).

What is the output?



```
console.log(2 == "2");  
console.log(2 === "2");
```

Ans: True, False

What is the output?



```
console.log(2 < "12");  
console.log("2" < "12");
```

Ans: True, False

What is the output?



```
const a = [1, 2, 3];  
const b = [4, 5];  
const c = [a, b];  
const d = [...a, ...b];  
console.log(c.length, d.length, d[3]);
```

Ans: 2, 5, 4

What is the output?



```
let x = 3;
x **= 2;
x %= 5;
console.log(x);
```

Ans: 4

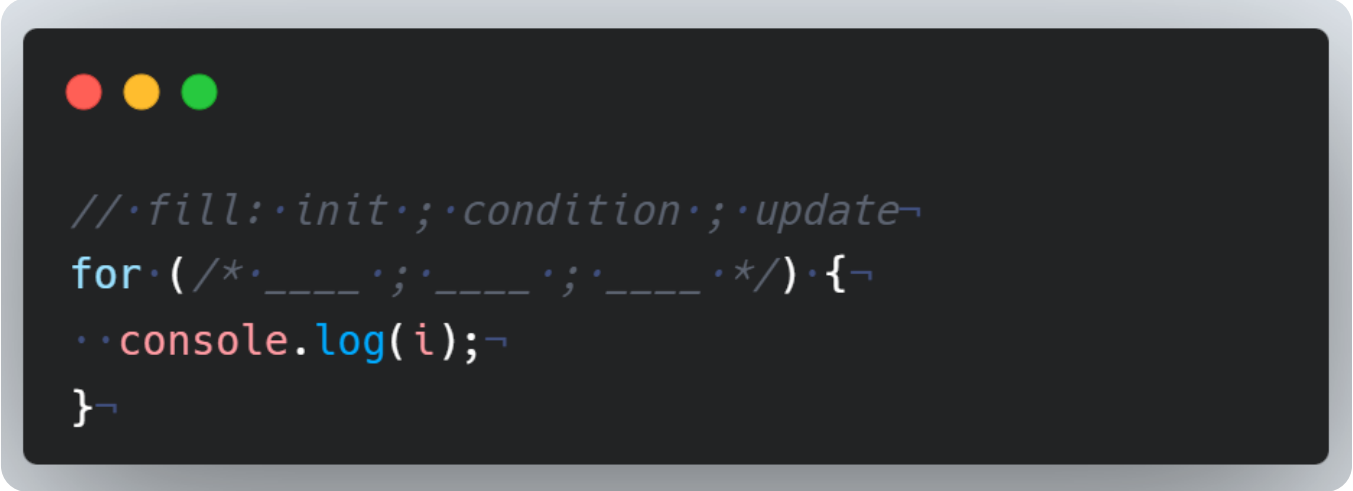
What is the output?



```
const a = null ?? "fallback";  
const b = 0 ?? 5;  
const c = "" || "fallback";  
const d = "" ?? "fallback";  
console.log(a, b, c, d);
```

Ans: fallback, 0, fallback, ""

Fill the blanks to print 5 → 1 using a for loop



```
// fill: init; condition; update  
for ( /* ____ ; ____ ; ____ */ ) {  
  console.log(i);  
}
```

Ans: *let i = 5; i >= 1; i--*

What prints?



```
const person = {  
  id: 101,  
  name: "Ari",  
  show() {  
    console.log(this.id, this.name);  
  }  
};  
const ref = person.show;  
ref();
```

Ans: undefined, undefined

What prints?



```
const a = [];  
a[2] = 99;  
a.forEach((v, i) => console.log(i, v));
```

Ans: 2,99

Merge arrays



```
const A = [1, 2, 3];  
const B = [4, 5];  
const merged = /* _____ */;  
console.log(merged);
```

Ans: const merged = [...A, ...B];

Thank You